The engineer of the near future

THE FRAME

From developer to forward deployed engineer · Tarugoconf 2026 · Matteo Di Paolantonio, CTO, Orbitant

> *We came to debate what an FDE is, and to show how we are modelling our own version so it fits a consultancy with no product behind it. Everything here comes from something we use.*

A founding engineer on someone else's product

"FDEs eat pain and excrete product." · Shyam Sankar, Palantir

- **What is not in the room.** No PM deciding scope, no staff engineer to escalate to, no backlog deciding what matters. You decide what to build, what to fake and what to push back on. Like a founding engineer, except the product is someone else's.
- **Own the problem end to end.** Understand the user, choose the scope, build, ship and follow through. We sold a SaaS that started as a hand-made PDF: validate the need, build as the business grows, choose deliberately what to fake.
- **A comb, not a T.** Engineering · AI stack · communication · product · business · domain. Enough breadth to connect everything.
- **FDE vs consultant.** Both need expertise, ingenuity and an understanding of the pain. Both should generalise what they learn. The difference is where the learning returns: to the product, or to the firm.
- **The analogy has a limit.** A founder owns the product and the roadmap. An FDE answers for the outcome inside someone else's authority.

Why now

- **Executing got cheap. Thinking did not.** Our engineering standard: about 4,000 words on executing a spec, one sentence on reaching it. The upstream part is the expensive one now.
- **The late question is the expensive one.** Mid-build, one question can change the data model, the logic and the designs: all of it paid in rework. Knowing the user's pain lets you ask it at minute 25, not mid-build.
- **An agent fills the gaps.** Every unstated assumption becomes a business decision nobody made. Resolve the ambiguity before delegating.
- **Works is not used.** OpenAI's FDE lead: weeks to solve the technical obstacle, months more of pilots, evals and trust.
- **The role is not new. What is scarce is** the part AI does not make cheaper: judgement, context, responsibility and adoption.

The seven criteria · our reading of the canon

Score your last project against them.

CRITERION	MEANING
01 · An outside client	A business outside your company, with its own P&L, constraints and politics.
02 · Business, direct	Direct access to the people who own the problem and the budget, nobody translating their pain.
03 · The end user	See the pain with the person who uses the solution; watch them work; learn what "solved" looks like to them.
04 · Build · skip · fake	Decide what to build, what to omit, and what to simulate for now with manual work or mocks.
05 · In production, used	Ship something somebody uses in their real workflow, not a pilot that dies in a drawer.
06 · Follow-up	Stay responsible after launch: monitor, iterate, fix what breaks, answer when it doesn't deliver.
07 · Learning returns	Feed what you learn back into the product, the platform or the next engagement.

Our version, for now

El ingeniero espabilado. Attitude can be nurtured. Aptitude can be improved. The environment grows both.

- **Attitude:** willingness to learn, investigate, try and fail. **Aptitude:** skills built through courses, training and real practice. **Environment:** time, access, feedback and ownership that let both grow.

We were already doing this before AI. Five things we did when code was expensive:

- **The client inside.** *"No sabemos trabajar sin alguien del cliente trabajando con nosotros."*
- **Production.** *"No hay valor si no está en producción."*
- **Ownership.** *"Da igual que seas el junior, el senior, el arquitecto o el staff."*
- **The best code.** *"El mejor código es el que no vas a escribir."*
- **Learning by failing.** *"Falla rápido, falla barato."*

AI Solutions Senior Engineer. Our job description so far, one word per pillar:

- **AI → innovation:** find your way: prototype, fake, probe. Agents, evals, MCP integrations.
- **Solutions → business and product:** own the solution, hence the problem, end to end. Scoping with the client, product trade-offs, value delivered.
- **Senior → the bar:** seniors only, for now. Judgement and autonomy at that level.
- **Engineer → engineering:** production-grade systems, from design to deployment and operation; evals, observability and traceability included.

IN PRACTICE

The daily check

1. What does it cost the business if I don't solve this?
2. What was I asked for, and what am I really solving?
3. Which question would change the scope, the solution or the outcome? Have I asked it?
4. Is this mine to decide, here? Push back, workaround, durable fix, or no?
5. Who owns it after launch, and how will I know it's used?

What to know, by area

AREA	STUDY FOCUS
Economics	Cost, margin, cash, business model.
Product	Outcomes, discovery, priorities, metrics.
Design	Experience, prototypes, validation.
Communication	Interviews, decisions, trade-offs.
AI	Agents, context, evaluation, oversight.
Technical	Architecture, tests, delivery, operation.
Innovation	Experiments that answer unknowns.
Back home	Knowledge others can apply.

WHAT TO STUDY

Product course · product, design, communication

OUTPUT, OUTCOME & ROADMAP

- **Output:** delivered work. **Outcome:** changed user or business results.
- **Lifecycle / roadmap:** discover, deliver, grow, retire; sequence problems and outcomes.

PRODUCT OPERATING MODEL

- Teams own problems and outcomes, with authority to choose solutions.
- Strategy → discovery → delivery → review: a recurring learning cycle.

PRODUCT TRIO

- PM + product designer + tech lead share discovery and delivery decisions.
- Cover value, usability, technical feasibility, business viability and ethical risk.

PM & TECH-LEAD COMPETENCIES

- **PM:** product execution, customer insight, strategy and influencing people.
- **Tech lead:** technology, product strategy, collaboration and leadership; connect architecture to outcomes.

PRIORITISATION & TECHNICAL DEBT

- **Prioritisation:** choose criteria and what to defer. RICE: reach, impact, confidence, effort.
- **Debt:** principal = repair cost; interest = recurring cost of leaving it.

PRODUCT METRICS

- **Leading:** early signals. **Lagging:** results observed later.
- **OMTM:** one metric that matters at this stage; measure behaviour and impact.

PRODUCT DISCOVERY · THE RIGHT PROBLEM

- Establish whose problem matters: align, research, define.
- **The Mom Test:** ask about past behaviour and real examples; avoid pitching for approval.

PRODUCT DISCOVERY · THE RIGHT SOLUTION

- Generate alternatives; identify the riskiest assumption and next learning goal.
- Validate cheaply with prototypes; deliver and measure the outcome.

Economics · Software Economics & finance

SALARY, COST & BILLABLE DAYS

- Employer cost includes salary, benefits and contributions; non-billable time also matters.
- Cost per billable day = relevant annual cost ÷ invoiceable days.

BILLING & OVERHEAD

- **Daily / monthly billing:** price capacity and allocate availability risk differently.
- **Overhead:** shared management, sales and operations costs, covered by delivery revenue.

GROSS MARGIN & EBITDA

- **Gross margin:** revenue less direct delivery costs, as an amount or percentage.
- **EBITDA:** earnings before interest, taxes, depreciation and amortisation; neither cash nor net profit.

BALANCE SHEET, P&L & CASH FLOW

- **Balance sheet:** assets, liabilities, equity at a date. **P&L:** revenue, expenses, profit over time.
- **Cash flow:** money entering and leaving; invoicing and collecting differ.

OPEX, CAPEX & VALUE

- **OPEX:** operating expense. **Capitalised development:** qualifying costs become an asset, then are amortised; cash is still spent.
- **Valuation:** enterprise value ≈ EBITDA × multiple is one approach.

AI & technical · Agentic development course

.MD FILES & CONTEXT

- A project map: architecture, conventions, commands, constraints.
- Maintain relevant context; separate unrelated tasks and preserve decisions at handoff.

AGENTS, SUBAGENTS & SKILLS

- **Agent:** uses tools towards a goal. **Subagent:** delegated task with its own context.
- **Skill:** reusable instructions or knowledge. Decompose work; review results yourself.

HOOKS & GUARDRAILS

- **Hook:** event-triggered automation; command hooks execute programmed checks.
- Enforce constraints in code and permissions; instructions alone cannot enforce them.

SPEC-DRIVEN DEVELOPMENT

- Interview → spec → plan → implement → review: define behaviour before code.
- Use for ambiguous, substantial changes; keep trivial edits lightweight.

PARALLEL WORK & INDEPENDENT REVIEW

- **Git worktree:** separate working directory and branch for independent work.
- Separate implementation and review contexts; verify the integrated result.

LEGACY CODE & TESTS

- **Characterisation tests:** capture existing behaviour before changing unfamiliar code.
- Check requirements and edge cases; passing tests cover only their assertions.

MCP & CLI

- **MCP:** a standard for exposing tools, resources and prompts to AI applications.
- **CLI:** command-line interface. Choose by task and capabilities; MCP can be local or remote.

AI & technical · Claude Certified Architect

AGENTIC ARCHITECTURE & ORCHESTRATION

- Design the model → tool → result loop, task decomposition and handoffs.
- Set stopping conditions, budgets and boundaries for autonomous action.

TOOL DESIGN & MCP INTEGRATION

- Define narrow capabilities: descriptions, input schemas, permissions, structured errors.
- Distinguish failure from empty results; make retries safe for side effects.

CLAUDE CODE CONFIGURATION & WORKFLOWS

- Scope project instructions, rules, skills and tool access to the work.
- Integrate plan, implementation, review and verification into CI/CD.

PROMPT ENGINEERING & STRUCTURED OUTPUT

- State the task, constraints, success criteria and examples.
- **Structured output:** schema-matching data; validate meaning and format.

CONTEXT MANAGEMENT & RELIABILITY

- Preserve decisions, sources and uncertainty through retrieval and handoffs.
- Test failures; escalate missing authority, unresolved ambiguity or explicit human requests.

WHAT TO FOSTER

In the team

THE CLIENT INSIDE

- Someone from the client working with the team, every day. Not one person holding the daily: every junior, mid and senior in front of them.

TIME TO THINK

- Profiles allowed to spend two or three days just thinking. An FDE we interviewed: a good platform team multiplies the FDEs' work by ten.

KNOWLEDGE SHARING SESSIONS

- Recorded, regular, open to the whole company. What one engineer learns at one client comes back as knowledge for the next.

PRODUCTION AS THE YARDSTICK

- The question is never "does it work?" but "is it used?". Follow-up is part of the job, not a maintenance contract.

Asteroids & Probes · innovation and back home

ASTEROIDS · WHAT THEY ARE

- A micro-session of knowledge, under 5 minutes, on any topic: AI, engineering, product, leadership.
- Four formats: live talk + recording, written post, annotated code, video. Anyone proposes one; merged, it lands in a searchable base. **Orbit:** a series when a topic needs depth.

PROBES · WHAT THEY ARE

- A long-range exploration: a question we can't answer today, a minimal route, and a commitment to report what turns up.
- Runs in the learning time Asteroids already use. Still a proposal, not yet a programme.

PROBES · ANATOMY

- **Mission:** what we want to know at the end that we don't know now. **Trajectory:** three milestones, corrected in flight.
- **Telemetry:** a dated logbook, dead ends included. **Payload:** format agreed at launch; the finding is not.

THREE VALID ENDINGS · LEARNING RETURNS

- *Sabemos hacerlo* (how, limits, cost) · *sabemos que no se puede* (why, what's ruled out) · *sabemos hasta dónde*.
- Probe → Asteroids → Orbit → knowledge sharing → article.